Back to index

Query Evaluation Techniques for Large Databases (7)

Goetz Graefe

Summary by: Steve Gribble and Armando Fox

One-line summary: A comparison of techniques, costs, and implications of sort and hashed based query processing algorithms is presented in section 7 of this paper.

Overview/Main Points

- **In-memory algorithm**: if data set fits in memory, quicksort is the sorting algorithm of choice, and classic in-memory hashing is the hashing technique of choice.
- **Divide-and-conquer paradigm**: in sorting, large data set is divided into subsets (physically - chunks as big as memory), and combined using (logical) merging. In hashing, a large data set is partitioned (logically) using hash values, and the partitions combined (physically) by simple concatenation.
- **I/O Patterns**: while writing runs after sorting, I/O is sequential write, but while merging sort runs, I/O is random read. While hash partitioning, I/O is random write, and while merging partitions, I/O is sequential read.
- **Temporary files accessed simultaneously**: amount of memory dictates merge fan-in for sorting (memory size / buffer space needed per run), and same fraction is fan-out for writing partition files.
- **I/O Optimizations**: Sorting can take advantage of read-ahead controlled by forecasting to reduce I/O delay. For hashing, can do write-behind of partitions. Also, double-buffering and striping can be used in both. Goal in both is to match load on I/O and CPU to keep system fully utilized.
- **Very large inputs**: For very large input sets, multi-level merging in sorting and hierarchical/recursive partitioning in hashing are used. There is some issue as to the optimality of the final merge in sort, and also the size of the leaf partitions in hashing; both need to maximally use memory, but naive recursion may not guarantee that the lowest level run sizes/partitions will do this. Merge optimizations (for sorting), and bucket-tuning and hybrid hash joins (for hashing) are techniques that will do this. Also, replacement selection (for sorting) is another optimization that makes better use of memory.
- **Aggregation/duplicate removal**: With hashing, if hash on the aggregation key, then aggregation/duplicate removal can be done within the hash buckets, and the operation's output may fit in memory, so classic hashing could be used. With sorting, if replacement-selection is used for run-generation, a similar trick can be played.
- **Algorithm phases**: For sorting, the algorithm proceeds in three phases: run generation, intermediate merging, and final merging. For hashing, the three phases are initial partitioning, intermediate partitioning, and hybrid/in-memory hash methods process partitions to produce output. If iterator interfaces are used for input and output, these phases may be executing in parallel, on demand.
- **Resource sharing**: Depth-first partitioning (for hashing, a dual to eager merging in sorting) implies the final phase of hashing may execute while the initial phases are not yet finished - this leads to poor memory utilization, and the final phase is done in-memory. Breadth-first partitioning (dual to lazy merging in sorting) avoids this.
- **Partitioning skew and effectiveness**: In hashing, if the hash function has skew, partitions of different sizes will be produced, which is bad. Similarly, merge run files of different sizes are bad for sorting.
- **Bit vector filtering**: If a one-to-one match operation is to be performed, a bit vector (large hash array) is used to early detect and reject items in the second input that cannot possibly have a match in the first input. Bit vectoring can be done at each level of multi-level partitioning or merging. It can also be used in both directions - to reduce the first input using a bit vector filter based on the second input.
- **Interesting orderings - multiple joins**: Merge-joins based on sorting output results in sorted order, so multiple merge-joins based on the sorted key can be more efficient. More difficult in hashing, as hash algorithms produce output in unpredictable order; process the N multiple inputs in parallel, effectively producing N-tuple partitions. Complex. Similar story with aggregation followed by a join, as sort-based aggregation again outputs data in sorted

order.

Relevance

Points out a large number of similarities and contrasts between sort and hash based query processing algorithms. Situations in one is better than another, and situations in which care must be taken for either to work well are detailed, which is helpful.

Flaws

- The logical organization of this section seems to be very meandering and without direction, which makes it hard to glean high-level concepts and implications.
- Which of the observations in this section are important and have impact/implications in the real world, and which are not important? It's hard to tell.

Back to index